Week 2: Assignment 2

From Spectral Analysis to Learned Representations

Assignment Focus: Connecting graph Laplacian eigenvalues to GNN message passing and learned embeddings
Due Date: End of Week by Thursday night (submit via WhatsApp)

1 Connecting to Assignment 1

Last week you built adjacency matrices, computed graph Laplacians, and analyzed eigenvalue distributions for different edge weighting schemes. This week, we connect those concepts to how GNNs actually learn.

The key insight: The graph Laplacian and GNN message passing are closely related. When a GNN updates node features by averaging over neighbors, it is applying a *diffusion operator* closely related to the Laplacian. The eigenvalues you computed tell us about information flow in the graph—and GNNs learn to exploit this structure.

Week 1 → Week 2 Connections

Assignment 1: What you built	**Assignment 2: How GNNs use it**
Adjacency matrix A	Defines which atoms send messages to which
Edge weights ($1/d$, Gaussian, etc.)	GNNs learn optimal weighting automatically
Laplacian $L = D - A$	Message passing $\approx$ repeated diffusion (see §3.1)
Laplacian eigenvalues	Determine how fast information spreads
Fiedler value λ_2 (algebraic connectivity)	Controls how fast diffusion ($H \leftarrow PH$) mixes node features $\Rightarrow$ heuristic guidance for GNN depth

2 Learning Objectives

By the end of this assignment, you should be able to:

- Explain how GNN message passing relates to the graph Laplacian
- Understand why GNNs need initial node features (embeddings)
- Extract and visualize learned element embeddings from pre-trained GNNs
- Connect embedding structure to the periodic table and known chemistry
- Compare what different GNN architectures learn
- Recognize statistical pitfalls in correlation analysis (multiple comparisons)

3 Key Concepts

3.1 From Laplacian to Message Passing

In Week 1, you computed $L = D - A$ and found its eigenvalues. Here's why that matters for GNNs.

Neighbor averaging is not directly applying the Laplacian—it is applying a **diffusion operator**. The two are closely related:

$$\text{Random-walk diffusion:} \quad P = D^{-1}A$$
$$\text{Random-walk Laplacian:} \quad L_{\text{rw}} = I - D^{-1}A = I - P$$

So repeated message passing (without nonlinearities) is basically repeated multiplication by P—or by $\tilde{D}^{-1/2}(A+I)\tilde{D}^{-1/2}$ in GCN-style normalization. Since $L_{\text{rw}} = I - P$, the eigenvectors are the same for both operators, and the eigenvalues are simply related: if λ is an eigenvalue of L_{rw}, then $1 - \lambda$ is the corresponding eigenvalue of P. The spectral analysis you did in Week 1 therefore directly describes how information propagates during message passing.

Convergence and the Fiedler value: The rate at which repeated diffusion (P^k) drives all node features toward a uniform state is governed by the second-largest eigenvalue of P, which equals $1 - \lambda_{\text{Fiedler}}$. A larger Fiedler value λ_{Fiedler} means the second-largest eigenvalue of P is smaller, so diffusion mixes faster and fewer message-passing layers are needed to spread information across the graph.

A general GNN layer adds learned weights W and a nonlinearity σ on top of this diffusion step:

$$h_i' = \sigma\left(W \cdot \sum_{j \in \mathcal{N}(i)} w_{ij} \cdot h_j\right)$$

To build intuition, we start with the simplified linear version: no learned weights, no nonlinearity, just degree-normalized averaging:

$$h_i' = \frac{1}{\text{degree}(i)} \sum_{j \in \mathcal{N}(i)} h_j = (D^{-1}A\,H)_i = (P\,H)_i$$

Why this simplification matters: Understanding the linear diffusion case gives you the spectral foundation. The learnable parameters W and nonlinearity σ then allow the GNN to selectively amplify or suppress different frequency components of the graph signal—going beyond what simple diffusion can achieve.

3.2 What are Embeddings?

In Week 1, your node features were just atom positions. But GNNs need richer starting information. An **embedding** is a learned vector that represents each element (H, He, Li, ...). The GNN learns these during training.

In the ALIGNN architecture we use here, the embedding is keyed solely by atomic number Z—the model receives no handcrafted chemical features like electronegativity or ionic radius.[1] If the model predicts properties accurately, it must have learned relevant chemistry implicitly from the training data (crystal structures and their target properties). We can check this by examining the embeddings.

[1] Other GNN architectures may augment or replace learned embeddings with handcrafted element feature vectors. For example, the original CGCNN uses a 92-dimensional handcrafted feature vector per element, and some versions of MEGNet also incorporate engineered features. Always check what your specific model uses as input.

3.3 CGCNN vs ALIGNN

	CGCNN (2018)	ALIGNN (2021)
Graph structure	Same as Week 1: atoms + bonds	Two graphs: atoms + bond angles
Edge weights	Learned from distances	Learned from distances AND angles
Accuracy	Good	Often significantly better†

†ALIGNN often improves over distance-only baselines on several JARVIS benchmark targets; the magnitude depends on the dataset, split, and target property. See the ALIGNN paper (Choudhary & DeCost, *npj Comput. Mater.* 2021) and the JARVIS Leaderboard for detailed comparisons.

Connection to Week 1: Remember how different edge weighting schemes ($1/d$, Gaussian, etc.) gave different eigenvalue distributions? CGCNN learns an optimal weighting. ALIGNN goes further by also encoding angular information—geometry that the simple adjacency matrix cannot capture.

4 Background Reading (1–2 hours)

- Review your Week 1 notes on the Laplacian and eigenvalues
- Watch CS224W Lecture 3: "Node Embeddings" (first 30 minutes)
- Skim CGCNN paper Section II (Methods): `https://arxiv.org/abs/1710.10324`

5 Hands-On Tasks

Version pinning: Before starting, record your environment for reproducibility:

```
pip freeze > requirements_frozen.txt
```

Submit this file alongside your other deliverables. If you encounter version-related errors, compare with classmates' frozen requirements.

Working directory: Run all scripts from a single project folder (e.g., `week2/`). Tasks 4 and 5 load the `.npy` files saved in Task 3, so all scripts should share the same working directory.

5.1 Task 1: Message Passing by Hand (Paper Exercise)

Time estimate: 20 minutes

Using your 4-atom square graph from Week 1 (with adjacency matrix A), perform one round of **simplified** message passing by hand—no learned weights, no nonlinearity, just diffusion via the operator $P = D^{-1}A$.

Initial node features (pretend these are element embeddings):

$$h_1 = [1, 0] \quad h_2 = [0, 1] \quad h_3 = [1, 1] \quad h_4 = [0, 0]$$

For each node, compute the new feature as the average of its neighbors' features:

$$h'_i = \frac{1}{\text{degree}(i)} \sum_{j \in \mathcal{N}(i)} h_j = (P\,H)_i$$

1. Write out h'_1, h'_2, h'_3, h'_4 after one message passing step.
2. Verify by matrix multiplication: compute $P \times H = (D^{-1}A) \times H$ where H is the 4×2 feature matrix. You should get the same result.

3. Perform a second round of message passing (apply the same operation to your updated features). Are the node features becoming more similar?
4. What happens after many rounds? The convergence rate is governed by $1 - \lambda_{\text{Fiedler}}$ (the second-largest eigenvalue of P). Look up the Fiedler value you computed for the square graph in Week 1—does the convergence speed you observe match what that eigenvalue predicts?

Deliverable: Handwritten work (`task1_handwork.pdf`)

5.2 Task 2: Environment Setup

Time estimate: 5 minutes

Add to your Week 1 environment:

```
conda activate gnn_crystals
pip install alignn mendeleev pandas
```

Deliverable: Screenshot showing successful import of alignn and mendeleev

5.3 Task 3: Extract Embeddings from Pre-trained Models

Time estimate: 15 minutes

Write a script that loads three pre-trained ALIGNN models and extracts the learned element embeddings.

Note: Depending on the ALIGNN version, `get_figshare_model` may return the model directly or a tuple `(model, config)`. The code below handles both cases. Always call `.eval()` to set the model to inference mode and use `.cpu()` to ensure tensors are on CPU before converting to NumPy.

```
import os
from alignn.pretrained import get_figshare_model
import numpy as np

# All outputs go to the current working directory
output_dir = os.path.dirname(os.path.abspath(__file__))

model_names = {
    'formation_energy': 'jv_formation_energy_peratom_alignn',
    'band_gap': 'jv_optb88vdw_bandgap_alignn',
    'bulk_modulus': 'jv_bulk_modulus_kv_alignn',
}

models = {}
for label, figshare_name in model_names.items():
    result = get_figshare_model(figshare_name)
    # Handle both (model, config) tuple and bare model returns
    model = result[0] if isinstance(result, tuple) else result
    model.eval()
    models[label] = model

for name, model in models.items():
    emb = model.atom_embedding.weight.detach().cpu().numpy()
    print(f'{name}: embedding shape = {emb.shape}')
    # Row 0 is typically a padding index; row i = element Z=i
    print(f'  Row 0 (padding): norm = {np.linalg.norm(emb[0]):.4f}')
    print(f'  Row 1 (H):       norm = {np.linalg.norm(emb[1]):.4f}')
```

```
    # Save for later tasks
    save_path = os.path.join(output_dir, f'embeddings_{name}.npy')
    np.save(save_path, emb)
    print(f'  Saved to {save_path}')
```

Indexing convention: ALIGNN's embedding table uses row 0 as a padding index. Element hydrogen (Z=1) maps to row 1, helium (Z=2) to row 2, and so on. You will use the slice `emb[1:N+1]` to extract embeddings for elements Z=1 through Z=N in later tasks. The print statements above let you verify this: the padding row often has a noticeably different norm.

Deliverable: Script (`task3_extract.py`) and output showing embedding dimensions

5.4 Task 4: Visualize Embeddings in 2D

Time estimate: 30 minutes

High-dimensional embeddings can be projected to 2D for visualization. Each student will use a **different** method so we can compare results:

	Faizan: t-SNE	**Ali: UMAP**
Install	(included in scikit-learn)	`pip install umap-learn`
Import	`from sklearn.manifold import TSNE`	`import umap`
Key params	`perplexity=15, random_state=42`	`n_neighbors=15, min_dist=0.1, random_state=42`

Both students should follow the same pipeline:

1. Take elements Z=1–80 from each model's embeddings (use rows `emb[1:81]`, skipping the padding row)
2. Apply PCA to reduce to 30 dimensions first (stabilizes both methods on high-dimensional input)
3. Reduce to 2D using your assigned method with the parameters above
4. Color elements by category using a simplified three-class scheme: **metals** (blue), **metalloids** (green), and **nonmetals** (red). Use the `mendeleev` library to look up element categories.
5. Label each point with the element symbol

Why two methods? t-SNE and UMAP both preserve local neighborhood structure, but they make different tradeoffs:

- **t-SNE** focuses on preserving local distances. Cluster shapes and inter-cluster gaps can be misleading—distances *between* clusters are not meaningful. It is also stochastic and sensitive to perplexity.
- **UMAP** better preserves global structure (relative positions of clusters) and is generally faster and more stable. The `n_neighbors` parameter plays a similar role to perplexity; `min_dist` controls how tightly points are packed.
- Both require `random_state` for reproducibility. Both benefit from PCA pre-reduction.

After both students have their plots, compare: Do the same element clusters appear in both visualizations? Where do they agree or disagree? Include 2–3 sentences of comparison in your deliverable.

Deliverable:

- ***Faizan***: `task4_tsne.py` and plots `tsne_formation_energy.png`, `tsne_band_gap.png`, `tsne_bulk_modulus.png`
- ***Ali***: `task4_umap.py` and plots `umap_formation_energy.png`, `umap_band_gap.png`, `umap_bulk_modulus.p`
- Both: 2–3 sentence comparison of the two methods' outputs

5.5 Task 5: Correlate with Physical Properties

Time estimate: 60 minutes

Test whether the learned embeddings encode known chemistry. For elements Z=1–50, correlate each embedding dimension with electronegativity, atomic radius, and valence electrons.

Important: Some elements have missing data (e.g., noble gases lack electronegativity values). Your code must handle NaN values. Also note the embedding indexing: use `emb[1:51]` to select rows for Z=1 through Z=50 (row 0 is a padding index).

Use the template below:

```
from alignn.pretrained import get_figshare_model
from mendeleev import element
from scipy.stats import pearsonr
import numpy as np

result = get_figshare_model('jv_formation_energy_peratom_alignn')
model = result[0] if isinstance(result, tuple) else result
model.eval()
emb = model.atom_embedding.weight.detach().cpu().numpy()

# Elements Z=1 to Z=50 -> rows 1 to 50 (skip padding row 0)
emb_elements = emb[1:51]

# Get properties (handle None -> NaN)
props = {'electronegativity': [], 'atomic_radius': [], 'valence': []}
for z in range(1, 51):
    el = element(z)
    en = el.electronegativity_pauling
    props['electronegativity'].append(en if en else np.nan)
    props['atomic_radius'].append(el.atomic_radius or np.nan)
    props['valence'].append(el.nvalence() or np.nan)

n_dims = emb_elements.shape[1]

# Find best correlation per property (skip NaN)
for prop_name, prop_vals in props.items():
    prop_arr = np.array(prop_vals)
    valid = ~np.isnan(prop_arr)
    best_r = 0
    for dim in range(n_dims):
        r, _ = pearsonr(emb_elements[:, dim][valid], prop_arr[valid])
        if abs(r) > abs(best_r): best_r = r
    print(f'{prop_name}:␣best␣R␣=␣{best_r:.3f}␣␣'
          f'(searched␣{n_dims}␣dims)')
```

Typical output ranges (your values will vary depending on model version and library updates):

```
electronegativity: R ~ 0.5 to 0.75
atomic_radius:     R ~ -0.4 to -0.7
valence:           R ~ 0.3 to 0.55
```

If your values fall outside these ranges, double-check your indexing: verify that row 1 of the embedding matrix corresponds to hydrogen (Z=1), not row 0.

Task 5b: Multiple Comparisons Control (Important!)

You just searched across *all* embedding dimensions and picked the best $|r|$. With dozens or hundreds of dimensions, you are virtually guaranteed a "good" correlation by chance—even with random data. This is the **multiple comparisons problem**.

To check whether your correlations are meaningful, add a permutation baseline. The permutation test must be run **inside** the property loop so that each property gets its own null distribution:

```
n_permutations = 1000
rng = np.random.default_rng(seed=42)

for prop_name, prop_vals in props.items():
    prop_arr = np.array(prop_vals)
    valid = ~np.isnan(prop_arr)
    valid_props = prop_arr[valid]
    valid_emb = emb_elements[:, :][valid]  # rows for valid elements

    # Observed best |r|
    best_r = 0
    for dim in range(n_dims):
        r, _ = pearsonr(valid_emb[:, dim], valid_props)
        if abs(r) > abs(best_r): best_r = r

    # Null distribution: permute property labels
    null_max_r = []
    for _ in range(n_permutations):
        perm = rng.permutation(valid_props)
        perm_best = 0
        for dim in range(n_dims):
            r, _ = pearsonr(valid_emb[:, dim], perm)
            if abs(r) > abs(perm_best): perm_best = r
        null_max_r.append(abs(perm_best))

    null_arr = np.array(null_max_r)
    p_value = np.mean(null_arr >= abs(best_r))
    print(f'{prop_name}: R = {best_r:.3f}, '
          f'permutation p = {p_value:.4f}, '
          f'null 95th pct = {np.percentile(null_arr, 95):.3f}')
```

If your observed $|r|$ is well above the 95th percentile of the null distribution, the correlation is likely real. If not, the model may not have strongly encoded that property.

Extend the full analysis (with permutation tests) to all three models and create a summary table with columns: model, property, best R, permutation p-value.

Deliverable: Script (`task5_correlate.py`) and correlation table showing R values and p-values for each model $\times$ property combination

5.6 Task 6: Structure Meets Identity—Connecting Eigenvalues and Embeddings

Time estimate: 45 minutes

A GNN prediction depends on two kinds of information: the *graph structure* (captured by Laplacian eigenvalues) and the *node identity* (captured by element embeddings). In this task, you will make that distinction concrete.

1. **Retrieve your Week 1 data.** Load the Laplacian eigenvalues you computed for the silicon crystal (Si, diamond cubic) from Week 1 Task 4.
2. **Build a second crystal graph.** Construct the crystal graph for germanium (Ge, also diamond cubic) using the same cutoff radius. Compute its Laplacian eigenvalues.
3. **Compare eigenvalue spectra.** Plot both eigenvalue distributions on the same axes. Since Si and Ge share the diamond cubic structure, the spectra should be similar (though not identical, due to different lattice constants).
4. **Extract embeddings.** Look up the learned embedding vectors for Si (Z=14) and Ge (Z=32) from the formation energy model. Use rows `emb[14]` and `emb[32]` respectively. Compute the cosine similarity between them.
5. **Synthesis.** Answer the following in 4–6 sentences:
   - Si and Ge have similar graph structure but different formation energies. What must the GNN rely on to distinguish them?
   - Now consider Si (diamond cubic) vs. Si_3N_4 (a structurally different silicon compound). Here the element is similar but the graph structure differs. What does each information source—eigenvalues vs. embeddings—contribute?
   - Why does a GNN need *both* structural and elemental information to make accurate predictions?

Deliverable: Script for eigenvalue comparison and embedding extraction (`task6_connect.py`), eigenvalue comparison plot (`task6_eigenvalue_comparison.png`), and written synthesis (`task6_synthesis.pd`

6 Questions for Reflection

1. In Week 1, you manually chose edge weighting schemes ($1/d$, Gaussian, etc.). How does a GNN's approach differ? What advantage does learning the weights provide?

2. The model was never explicitly given chemical properties like electronegativity. Based on your correlation results (including the permutation test), did it learn chemistry anyway? How might it have discovered this from crystal structures and target properties alone?

3. If you trained two GNNs—one to predict band gap and one to predict bulk modulus—would you expect them to learn identical element embeddings? Why or why not?

4. The Fiedler value λ_2 (algebraic connectivity) is the spectral gap that controls how fast diffusion / random-walk averaging mixes on a graph. Since message passing is a form of diffusion, how can λ_2 guide a *heuristic* choice of GNN depth (number of layers)?

5. What information is captured in the learned embeddings that is NOT captured in the Laplacian eigenvalues? What does the Laplacian capture that embeddings don't?

7 Submission

Submit a single folder named `assignment2_yourname/` containing:

File	Contents
`assignment2_report.pdf`	All written work: Task 1 handwork, Task 2 screenshot, Task 4 plots + comparison, Task 6 eigenvalue plot + synthesis, reflection questions
`task3_extract.py`	Embedding extraction script
`task4_tsne.py` and `task4_umap.py`	Visualization script
`task5_correlate.py`	Correlation analysis + permutation test
`task6_connect.py`	Eigenvalue comparison + embedding extraction
`requirements_frozen.txt`	`pip freeze` output

Zip the folder and post to the WhatsApp group by Friday night.

8 Resources

- CS224W Lecture 3: `https://www.youtube.com/watch?v=rMq21iY61SE`
- ALIGNN paper: Choudhary & DeCost, *npj Comput. Mater.* (2021)
- ALIGNN GitHub: `https://github.com/usnistgov/alignn`
- JARVIS Leaderboard: `https://pages.nist.gov/jarvis_leaderboard/`
- Mendeleev library: `https://mendeleev.readthedocs.io`
- Spectral Graph Theory overview: `https://arxiv.org/abs/1609.08072`

9 Preview: Week 3

Next week we will: We move from analyzing *pre-trained* models to building and explaining. Each student will design a **publishable mechanistic interpretability study** using crystal GNNs.

The Core Question

GNNs for materials achieve high accuracy, but *what have they actually learned?* Weeks 1–2 gave you the tools: spectral analysis, embedding extraction, correlation testing with statistical controls. Now you will use these (and new ones) to answer a specific, novel interpretability question.*Do CGCNN, ALIGNN, MACE, and GemNet learn the same chemistry? What does each architecture uniquely capture? For which crystal families do GNNs systematically fail?*